model/01/app/models/project.rb
```
class Project < ApplicationRecord
  include Sizeable
```

There are a couple of other ways to invoke a shared example group—RSpec defines synonymous methods include_examples and it_behaves_like. You can also use RSpec metadata, which I'll discuss in Chapter 16, *Running Tests Faster and Running Faster Tests*, on page 319.

When invoking the group, you can also have the it_should_behave_like method take a block argument. Inside that block, you can use let statements to define variables, which are then visible to the shared example specs. In other words, an alternative to creating an instance with described_class is to place the burden on the calling spec to create a variable and give it an appropriate name in a let block, like this:

```
it_should_behave_like "sizable" do
  let(:instance) { Project.new }
end
```

Writing Your Own RSpec Matchers

RSpec's built-in matchers are flexible, but sometimes you have behavior patterns you specify multiple times in your code and the existing matchers don't quite cover it. Sometimes this is because the specification requires multiple steps, and sometimes it's because the generic matcher doesn't quite match the code's intent.

RSpec provides tools for creating your own custom matchers to cover just such an eventuality. A basic matcher is really simple. Let's say you wanted a custom matcher to measure project size in points. Rather than say expect(project.size).to eq(5) you'd say expect(project).to be_of_size(5). It's a little contrived, but work with me.

Normally custom matchers are placed in the spec/support folder, which can be imported when RSpec starts. To import your matcher, you need to explicitly require the custom matcher file in your rails_helper file. RSpec 3.1 contains a commented line in the rails_helper that you can uncomment in your setup to have the entire spec/support directory loaded automatically. You can also choose to directly import individual matcher files at the beginning of the spec files that use them.